Data Challenge Cheatsheet：pandas / numpy / plot / modelling

环境：Python 3.8.7 · Jupyter · numpy / pandas / scikit-learn。下面的代码都在 Python 3.8 + pandas 1.1.5 + sklearn 0.23.2 + statsmodels 0.12.1 下用模拟数据跑通过。

0. 环境

新写法	旧环境可能要写
`freq="h"`	`freq="H"`（下文统一用 `"H"`）
`pd.to_datetime(..., format="mixed")`	pandas < 2.0 没有这个参数，按格式分开解析（见 4.）
`OneHotEncoder(sparse_output=False)`	`OneHotEncoder(sparse=False)`
`.get_feature_names_out()`	sklearn < 1.0：`.get_feature_names(cat)`
`links.Probit()` / `links.Log()`	`links.probit()` / `links.log()`
`mean_squared_error(..., squared=False)`	统一写 `mean_squared_error(...) ** 0.5`
`df.mean(numeric_only=True)`	先 `df.select_dtypes("number")` 再算
Python 3.9+ 语法 `d1 | d2`、`str.removeprefix`、`list[int]`	都不能用

```
import sys
import numpy as np
import pandas as pd
import matplotlib
import matplotlib.pyplot as plt
import sklearn
print(sys.version, np.__version__, pd.__version__, matplotlib.__version__, sklearn.__version__)
pd.set_option("display.max_columns", 100)
pd.set_option("display.width", 200)
```

```
%pip install holidays
```

装完要 Restart kernel。`%` 后面不能有空格；也可以用 `!{sys.executable} -m pip install holidays`。提交前把这个 cell 删掉。

1. 读数 & 初探

```
df = pd.read_csv("ts.csv", parse_dates=["ts"])
print(df.shape)
print(df.head())
df.info()
print(df.describe(include="all").T)
print(df.isna().sum())
print(df.isna().mean().sort_values(ascending=False))
print(df.duplicated().sum(), df["ts"].duplicated().sum())
print(df.nunique())
print(df["cat"].value_counts(dropna=False))
print(df.select_dtypes("number").columns.tolist(), df.select_dtypes("object").columns.tolist())
print(df["y"].quantile([0.001, 0.01, 0.5, 0.99, 0.999]))
print(df.select_dtypes("number").skew())
print(df.select_dtypes("number").corr().round(2))
print(df["ts"].diff().value_counts().head())
print(df["ts"].is_monotonic_increasing)
```

2. 时间索引：DST → UTC → 补齐小时

```
df = df.drop_duplicates().sort_values("ts").reset_index(drop=True)
df["ts"] = df["ts"].dt.tz_localize("Europe/Berlin", ambiguous="infer").dt.tz_convert("UTC")
df = df.set_index("ts").asfreq("H")
print(df.index.min(), df.index.max(), len(df))
miss = pd.date_range(df.index.min(), df.index.max(), freq="H").difference(df.dropna(how="all").inde
x)
print(len(miss))
```

3. 清洗（时序）：异常值 → mark → 只用过去填补，并生成清洗记录表

顺序：先把异常值设成 NaN，再建 `mark`，最后 fill。

```
log = []
b = (df["y"] < 20000) | (df["y"] > 100000)
log.append(["y", "0 or ~10x", "set NaN", b.sum()])
df.loc[b, "y"] = np.nan
b = df["price"] >= 3000
log.append(["price", "9999 placeholder", "set NaN", b.sum()])
df.loc[b, "price"] = np.nan
g = (df["gen"].diff() != 0).cumsum()
b = df["gen"].groupby(g).transform("size") >= 6
log.append(["gen", "stuck 6h+", "set NaN", b.sum()])
df.loc[b, "gen"] = np.nan
df["cat"] = df["cat"].str.strip().str.lower()
mark = df[["y", "temp", "gen", "price"]].isna().any(axis=1)
log.append(["all", "missing", "fill lag168 + ffill, drop from train", mark.sum()])
df = df.fillna(df.shift(168)).ffill()
print(pd.DataFrame(log, columns=["col", "issue", "rule", "rows"]))
```

另外两种常见修正：

- 单位错误：`df.loc[b, "t"] = (df.loc[b, "t"] - 32) * 5 / 9`
- 整天放大 1000 倍：`df["s"].groupby(day).transform("max") > 1e5`

3b. 转回当地时间，join 第二个文件，取日历属性

```
df.index = df.index.tz_convert("Europe/Berlin")
t = pd.read_csv("fc.csv", parse_dates=["target_time"])
t["target_time"] = t["target_time"].dt.tz_localize("Europe/Berlin", ambiguous="infer")
df = df.join(t.set_index("target_time")["temp_fc"], how="outer")
idx = df.index
print(idx.hour[:3], idx.dayofweek[:3], idx.month[:3], idx.year[:3], idx.date[:3], idx.dayofyear[:
3])
print(idx.tz_localize(None).strftime("%Y-%m-%d %H:%M")[:3])
```

4. 清洗（横截面）：类型、文本、日期，train 和 test 一起做逐行操作

`drop_duplicates` 只能对 train 做，不要对拼起来的表做，否则会删掉 test 的行。

```
data = pd.read_csv("train.csv").drop_duplicates()
test = pd.read_csv("test.csv")
print(data["g"].value_counts(dropna=False).to_string())
print(data.loc[~data["inc"].astype(str).str.match(r"^\d+(\.\d+)?$", na=False), "inc"].unique()[:5])
print(data["d"].str.len().value_counts())
```

```
al = pd.concat([data, test], keys=["tr", "te"])
al["inc"] = pd.to_numeric(al["inc"].str.replace(r"[$,]", "", regex=True), errors="coerce")
al.loc[~al["age"].between(5, 100), "age"] = np.nan
al["g"] = al["g"].str.strip().str.lower().map({"male": "m", "m": "m", "female": "f", "f": "f"})
sl = al["d"].str.contains("/", na=False)
al["d"] = pd.to_datetime(al["d"].where(~sl), format="%Y-%m-%d", errors="coerce").fillna(pd.to_datet
ime(al["d"].where(sl), format="%m/%d/%Y", errors="coerce"))
al["days"] = (pd.Timestamp("2024-01-01") - al["d"]).dt.days
al["mon"] = al["d"].dt.month
al["cpy"] = al["calls"] / (al["ten"] / 12 + 1)
al["linc"] = np.log1p(al["inc"])
for c in ["age", "inc"]:
    al[c + "_na"] = al[c].isna().astype(int)
al = al.drop(columns=["d"])
data, test = al.loc["tr"].reset_index(drop=True), al.loc["te"].reset_index(drop=True)
print(data.isna().sum(), data["g"].value_counts(dropna=False))
```

标签清洗：先 `map`，再 `dropna`，最后才能 `astype(int)`。

```
df["y"] = df["y"].astype(str).str.strip().str.lower().map({"yes": 1, "y": 1, "no": 0, "n": 0})
df = df.dropna(subset=["y"])
df["y"] = df["y"].astype(int)
```

5. pandas 方法速查

方法	要点
`.asfreq("H")` / `.shift(n)`	先 asfreq，shift 才是真正的 n 小时
`.fillna(df.shift(168)).ffill()`	只用过去的数据；`interpolate` 会用到缺口之后的值，有 look-ahead
`.isna().any(axis=1)`	得到布尔 mark；`df[mask]` 得到的是数据，不能取反
`.reindex(idx, fill_value=False)` / `.reindex(columns=a.columns, fill_value=0)`	按 index 或列对齐
`.groupby([a, b])["y"].mean().unstack()`	结果是二维表

方法	要点
`.groupby(g).transform("max")`	返回原长度，可以直接当 mask
`.resample("D").mean()` / `.rolling(30).mean()`	日均 / 滚动
`.diff()` `.pct_change()` `.cumsum()` `.rank()`	时序常用
`pd.cut(s, bins)` / `pd.qcut(s, 5, labels=False)`	分箱
`pd.concat([...], axis=1)` / `.join(o, how="outer")` / `pd.merge(a, b, on="k", how="left")`	拼接 / 按 index / 按列
`.rename("x")` / `.rename(columns={})` / `.add_prefix("x_")`	避免列名重复
`.filter(like="gen_")` / `.filter(regex="^h_")`	选列
`.mul(s, axis=0)`	逐行乘，做交互项
`.isin([12, 1, 2])` / `(a) | (b)` / `(a) & (b)`	不能用 `or` / `and`
`df.loc[cond, "c"] = v`	不要链式赋值
`.between(lo, hi)` / `.clip(lo, hi)`	遇到 NaN 返回 False / 压到边界
`.str.strip().str.lower()` / `.str.replace(r"..", "", regex=True)` / `.str.match(..., na=False)` / `.str.contains`	文本
`map` vs `replace`	map 查不到变 NaN；replace 保留原值
`pd.to_numeric(errors="coerce")` / `pd.to_datetime(format=, errors="coerce")`	类型
`.dt.days` `.dt.month` `.dt.hour` `.dt.dayofweek`	时间访问器
`.to_period("W")` / `pd.period_range(..., freq="M")` / `str(mo - 1)`	按周、月分组和切片
`.idxmin()` / `.idxmax()` / `.nlargest(5, "c")`	返回 index，不是值本身
`s.autocorr(k)` / `.corr()`	自相关、相关矩阵
`.loc["2024-04-01":"2024-09-30"]`	两端都包含
`.iloc[:n]`	右端不包含
`pd.crosstab(a, b, normalize="index")` / `.value_counts(normalize=True)`	类别比例
`.select_dtypes("number")` / `.mode().iloc[0]`	选数值列 / 众数

5b. groupby / pivot / 多列操作

下面的例子都用这张示例表：

```
r = np.random.RandomState(0)
n = 1000
g = pd.DataFrame({"date": pd.date_range("2024-01-01", periods=n, freq="H"), "sec": r.choice(["tec
h", "fin", "energy"], n), "reg": r.choice(["US", "EU"], n), "ret": r.normal(0, 1, n), "vol": r.rand
int(100, 1000, n), "px": r.uniform(10, 100, n)})
```

groupby 方法速查

方法	返回	用途
`.mean()` `.sum()` `.std()` `.min()` `.max()` `.median()` `.first()` `.last()`	每组一行	基本聚合

方法	返回	用途
`.size()` vs `.count()`	每组一行	size 数行数（包括 NaN）；count 数非缺失值
`.nunique()`	每组一行	每组有几种不同的值
`.agg([...])` / `.agg({col: ...})` / `.agg(新名=(列, 函数))`	每组一行	多个函数、不同列用不同函数、命名聚合
`.quantile(q)`	每组一行	分位数
`.transform(f)`	**原长度**	组内去均值、z-score、占比，可以直接作为新列
`.rank(pct=True)` `.shift(1)` `.diff()` `.cumsum()` `.cumcount()`	原长度	组内排名、组内 lag、组内累计、组内序号
`.filter(lambda x: ...)`	原表的子集	保留满足条件的整组
`.apply(f)`	任意形状	最灵活，但最慢
`.head(k)` / `.tail(k)` / `.nth(k)`	原表的子集	每组前 k 行、后 k 行、第 k 行
`as_index=False` / `.reset_index()`	DataFrame	分组键变成普通列
`pd.Grouper(key="date", freq="W")`	—	按时间频率分组，可以和其他键一起用

一个键、多个键、多列

```
print(g.groupby("sec")["ret"].mean())
print(g.groupby(["sec", "reg"])["ret"].mean())
print(g.groupby(["sec", "reg"], as_index=False)["ret"].mean())
print(g.groupby("sec")[["ret", "vol"]].mean())
print(g.groupby("sec").size(), g.groupby("sec")["ret"].count(), g.groupby("sec")["reg"].nunique())
```

agg：多个函数、每列不同函数、命名聚合

```
print(g.groupby("sec")["ret"].agg(["mean", "std", "min", "max", "count"]))
print(g.groupby("sec").agg({"ret": ["mean", "std"], "vol": "sum", "px": "last"}))
a = g.groupby(["sec", "reg"]).agg(m=("ret", "mean"), s=("ret", "std"), v=("vol", "sum"), n=("ret",
"size")).reset_index()
print(a)
a = g.groupby("sec").agg({"ret": ["mean", "std"], "vol": "sum"})
a.columns = ["_".join(c) for c in a.columns]
print(a)
print(g.groupby("sec")["ret"].agg(lambda x: x.quantile(0.9)))
print(g.groupby("sec")["ret"].quantile([0.1, 0.9]).unstack())
```

- 命名聚合 `新列名=(原列, 函数)` 输出的列名最干净，推荐优先用。
- `agg({col: [f1, f2]})` 的结果是两层列名，用 `"_".join` 压平成 `ret_mean` 这种。

transform 和组内操作：结果和原表一样长

```
g["dm"] = g["ret"] - g.groupby("sec")["ret"].transform("mean")
g["z"] = (g["ret"] - g.groupby("sec")["ret"].transform("mean")) / g.groupby("sec")["ret"].transform
("std")
g["rk"] = g.groupby("sec")["ret"].rank(pct=True)
g["lag1"] = g.groupby("sec")["ret"].shift(1)
g["cs"] = g.groupby("sec")["vol"].cumsum()
g["k"] = g.groupby("sec").cumcount()
g["ma"] = g.groupby("sec")["ret"].transform(lambda x: x.rolling(5).mean())
g["sh"] = g["vol"] / g.groupby(["sec", "reg"])["vol"].transform("sum")
print(g.head(8))
```

- 组内 `shift` 和 `rolling` 不会跨组，也就是不会把上一只股票的数据带进下一只。面板数据做 lag 一定要这样写。
- 在 train/test 上用 `transform("mean")` 之类的统计量时，算出的是"整组"的统计量。建模时必须只用 train 来算，否则有泄漏。

filter / apply / 每组前几名

```
print(g.groupby("sec").filter(lambda x: len(x) > 300)["sec"].unique())
print(g.groupby("sec").apply(lambda x: x.nlargest(2, "ret"))[["sec", "ret"]])
print(g.groupby("sec").head(2))
print(g.sort_values("ret").groupby("sec").tail(1))
```

时间分组：resample / Grouper / ohlc

```
print(g.set_index("date").groupby("sec")["ret"].resample("D").mean().unstack(0).head())
print(g.groupby([pd.Grouper(key="date", freq="W"), "sec"])["vol"].sum().unstack().head())
print(g.set_index("date").resample("D").agg({"ret": "mean", "vol": "sum", "px": "last"}).head())
print(g.set_index("date")["px"].resample("D").ohlc().head())
```

pivot_table / crosstab

```
print(g.pivot_table(index="sec", columns="reg", values="ret", aggfunc="mean"))
print(g.pivot_table(index="sec", columns="reg", values=["ret", "vol"], aggfunc={"ret": "mean", "vo
l": "sum"}))
print(g.pivot_table(index="sec", columns="reg", values="ret", aggfunc=["mean", "count"], margins=Tr
ue))
print(g.pivot_table(index=g["date"].dt.hour, columns=g["date"].dt.dayofweek, values="vol", aggfunc
="mean").shape)
print(pd.crosstab(g["sec"], g["reg"]), pd.crosstab(g["sec"], g["reg"], normalize="index").round(2))
print(pd.crosstab(g["sec"], g["reg"], values=g["ret"], aggfunc="mean"))
```

- `pivot_table` 等于 `groupby([index, columns]).agg().unstack()`，会自动聚合重复的组合。
- `margins=True` 加上 "All" 行和列（总计）。
- `crosstab` 默认统计次数；`normalize="index"` 把每行变成比例。

长宽表互转：pivot / melt / stack / unstack

```
w = g.groupby(["sec", "reg"])["ret"].mean().reset_index()
wd = w.pivot(index="sec", columns="reg", values="ret")
print(wd)
lg = wd.reset_index().melt(id_vars="sec", var_name="reg", value_name="ret")
print(lg)
print(wd.stack().head(), w.set_index(["sec", "reg"])["ret"].unstack("reg"))
```

方法	方向	注意
`pivot(index, columns, values)`	长 → 宽	组合有重复会报错，那时改用 `pivot_table`
`melt(id_vars, var_name, value_name)`	宽 → 长	画图（按类别分组）、合并时常用
`unstack(level)`	行 index 的某一层 → 列	默认是最内层
`stack()`	列 → 行 index 的最内层	是 unstack 的反操作

其他可以传多列的地方

```
print(g.sort_values(["sec", "ret"], ascending=[True, False]).head())
print(g.drop_duplicates(subset=["sec", "reg"]).shape)
print(g.dropna(subset=["lag1", "ma"]).shape)
g2 = g.fillna({"lag1": 0, "ma": g["ma"].median()})
print(g[["sec", "reg", "ret"]].head(2), g.loc[g["ret"] > 2, ["sec", "ret"]].head(2))
o = pd.DataFrame({"sec": ["tech", "fin"], "reg": ["US", "EU"], "beta": [1.2, 0.8]})
print(g.merge(o, on=["sec", "reg"], how="left")["beta"].isna().mean())
o2 = o.rename(columns={"sec": "sector", "reg": "region"})
print(g.merge(o2, left_on=["sec", "reg"], right_on=["sector", "region"], how="inner").shape)
print(g.set_index(["sec", "reg"]).sort_index().loc[("tech", "US")].shape)
print(g.assign(lr=np.log(g["px"]), big=g["vol"] > 500).head(2))
print(g[["ret", "vol", "px"]].agg(["mean", "std"]))
print(g[["ret", "px"]].apply(lambda c: c.max() - c.min()))
print(g[["vol", "px"]].clip(lower=g[["vol", "px"]].quantile(0.01), upper=g[["vol", "px"]].quantile
(0.99), axis=1).max())
print(g.nlargest(3, ["ret", "vol"])[["ret", "vol"]])
print(g.value_counts(["sec", "reg"]))
print(g[["ret", "vol", "px"]].corr(method="spearman").round(2))
```

写法	多列的意思
`sort_values([a, b], ascending=[True, False])`	先按 a 升序，a 相同再按 b 降序
`drop_duplicates(subset=[a, b])` / `dropna(subset=[...])`	只看这几列来判断
`fillna({col: v})`	每列用不同的值填
`merge(on=[a, b])` / `left_on=[...], right_on=[...]`	多个键一起匹配
`set_index([a, b])` + `.loc[(x, y)]`	MultiIndex 按多个键取行
`df[[a, b]].agg([...])` / `.apply(f)`	每列分别算
`clip(lower=Series, upper=Series, axis=1)`	每列用自己的上下界
`nlargest(k, [a, b])` / `value_counts([a, b])`	多列排序 / 多列组合计数

6. numpy 速查

方法	用途
`np.log1p` / `np.expm1`	右偏且含 0 的列 / 反变换
`np.where(c, a, b)` / 嵌套 `np.where`	打标签
`np.clip(p, 0, 1)`	预测值裁剪
`np.logspace(-2, 4, 25)` / `np.linspace` / `np.arange`	参数网格
`np.corrcoef(a, b)[0, 1]`	相关系数
`np.sqrt(((y - p) ** 2).mean())`	RMSE
`np.exp(coef)`	logit → odds ratio；log link → 乘数效应
`np.percentile(a, [1, 99])` / `np.isnan` / `np.nan`	分位数 / 缺失
`np.random.default_rng(42)`；旧版用 `np.random.RandomState(42)`	随机数
`a[:, 1]`	二维切片，例如 `predict_proba` 取第 1 列

7. 画图

日均值 + 30 日均线

```
s = df["y"].loc[:"2024-09-30"]
dl = s.resample("D").mean()
ax = dl.plot(figsize=(14, 4), alpha=0.4, label="daily")
dl.rolling(30).mean().plot(ax=ax, label="30d mean")
ax.legend()
plt.show()
```

热力图：小时 × 星期几

```
m = df.groupby([idx.hour, idx.dayofweek])["y"].mean().unstack()
plt.imshow(m, aspect="auto", cmap="viridis")
plt.colorbar(label="y")
plt.xticks(range(7), ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"])
plt.xlabel("day of week")
plt.ylabel("hour")
plt.show()
```

日内曲线：按季节、工作日 vs 周末

```
se = pd.Series(idx.month % 12 // 3, index=idx).map({0: "winter", 1: "spring", 2: "summer", 3: "autu
mn"})
wd = np.where(idx.dayofweek < 5, "weekday", "weekend")
fig, ax = plt.subplots(1, 2, figsize=(14, 4), sharey=True)
df.groupby([idx.hour, se])["y"].mean().unstack().plot(ax=ax[0], title="by season")
df.groupby([idx.hour, wd])["y"].mean().unstack().plot(ax=ax[1], title="weekday vs weekend")
plt.show()
```

按组着色的散点图

```
dd = df[["y", "temp"]].loc[:"2024-09-30"].resample("D").mean()
plt.figure(figsize=(7, 5))
for k, gg in dd.groupby(dd.index.year):
    plt.scatter(gg["temp"], gg["y"], s=6, label=k)
plt.xlabel("temp")
plt.ylabel("y")
plt.legend()
plt.show()
```

ACF

```
ac = [s.autocorr(k) for k in range(1, 193)]
plt.figure(figsize=(12, 4))
plt.plot(range(1, 193), ac)
for k in [24, 48, 168]:
    plt.axvline(k, ls="--", c="grey")
plt.title("ACF")
plt.show()
```

缺失时间线（用未清洗的原始数据）

```
raw = pd.read_csv("ts.csv", parse_dates=["ts"]).drop_duplicates().sort_values("ts")
raw["ts"] = raw["ts"].dt.tz_localize("Europe/Berlin", ambiguous="infer").dt.tz_convert("UTC")
raw = raw.set_index("ts").asfreq("H")
plt.figure(figsize=(14, 4))
plt.imshow(raw.isna().T, aspect="auto", interpolation="none", cmap="Greys")
plt.yticks(range(raw.shape[1]), raw.columns)
xt = np.linspace(0, len(raw) - 1, 8).astype(int)
plt.xticks(xt, raw.index[xt].strftime("%Y-%m"))
plt.show()
```

直方图、箱线图、类别条形图

```
fig, ax = plt.subplots(1, 3, figsize=(15, 4))
df["price"].hist(bins=100, ax=ax[0])
df.boxplot(column="y", by=idx.month, ax=ax[1])
df["cat"].value_counts().plot.bar(ax=ax[2])
plt.tight_layout()
plt.show()
print(df["y"].describe(), np.corrcoef(df["y"].loc[:"2024-09-30"], df["gen"].loc[:"2024-09-30"])[0,
1])
```

8. 特征工程：dummies、交互项、节假日安全的 lag、滚动特征

```
import holidays
H = holidays.Germany(years=range(2021, 2026))
hol = pd.Series([d in H for d in idx.date], index=idx, name="hol").astype(int)
off = ((idx.dayofweek >= 5) | (hol == 1)).astype(int)
h = pd.get_dummies(idx.hour, prefix="h", drop_first=True).set_index(idx).astype(int)
dow = pd.get_dummies(idx.dayofweek, prefix="dow", drop_first=True).set_index(idx).astype(int)
mo = pd.get_dummies(idx.month, prefix="m", drop_first=True).set_index(idx).astype(int)
yr = pd.get_dummies(idx.year, prefix="yr", drop_first=True).set_index(idx).astype(int)
hw = h.mul(off, axis=0).add_prefix("off_")
hs = hol.astype(bool)
lag48 = df["y"].shift(48).where(~hs.shift(48, fill_value=False), df["y"].shift(216)).rename("lag4
8")
lag168 = df["y"].shift(168).where(~hs.shift(168, fill_value=False), df["y"].shift(336)).rename("lag
168")
r7 = df["y"].shift(48).rolling(168).mean().rename("r7")
X = pd.concat([h, dow, mo, yr, hw, hol, lag48, lag168, r7, df["temp_fc"], df["y"]], axis=1)
print(X.columns.is_unique, X.shape)
```

- 线性模型：用 dummy，并加 `drop_first=True`。
- 树模型：日历变量可以直接放整数。
- 滚动特征必须先 `shift` 再 `rolling`。
- 每个特征都要过信息集检查：预测时刻它已经知道了吗？

9. 切分

按时间切 train / valid / test

```
Xte = X.loc["2024-10-01":].drop(columns="y")
mk = mark.reindex(X.index, fill_value=False).astype(bool)
D = X.loc[~mk].dropna()
tr = D.loc[:"2024-03-31"]
va = D.loc["2024-04-01":"2024-09-30"]
Xtr, ytr = tr.drop(columns="y"), tr["y"]
Xva, yva = va.drop(columns="y"), va["y"]
Xall, yall = D.drop(columns="y"), D["y"]
print(Xtr.shape, Xva.shape, Xte.shape)
```

随机切（横截面数据，分类用 stratify）

```
X = data.drop(columns=["id", "out", "frac"])
y = data["out"].astype(int)
Xte = test.drop(columns=["id", "out", "frac"])
drop = [c for c in X if X[c].isna().mean() > 0.5 or X[c].nunique() <= 1]
X, Xte = X.drop(columns=drop), Xte.drop(columns=drop)
from sklearn.model_selection import train_test_split
Xtr, Xva, ytr, yva = train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)
print(ytr.mean(), yva.mean(), drop)
```

回归任务想分层，先分箱：`stratify=pd.qcut(y, 5, labels=False)`。

手写统计性处理（统计量只从 train 算）：填补、裁剪、get_dummies 后 reindex

```
num = Xtr.select_dtypes("number").columns.tolist()
cat = Xtr.select_dtypes("object").columns.tolist()
fv = pd.concat([Xtr[num].median(), Xtr[cat].mode().iloc[0]])
a = Xtr.fillna(fv)
b = Xva.fillna(fv)
lo, hi = a[num].quantile(0.01), a[num].quantile(0.99)
a[num] = a[num].clip(lo, hi, axis=1)
b[num] = b[num].clip(lo, hi, axis=1)
a = pd.get_dummies(a, columns=cat, dtype=int)
b = pd.get_dummies(b, columns=cat, dtype=int).reindex(columns=a.columns, fill_value=0)
print(a.isna().sum().sum(), b.isna().sum().sum(), a.columns.equals(b.columns))
```

按月 walk-forward（扩展窗口）

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
ps = {"naive": [], "ols": []}
for mo_ in pd.period_range("2024-04", "2024-09", freq="M"):
    a_ = D.loc[:str(mo_ - 1)]
    b_ = D.loc[str(mo_)]
    ps["naive"].append(b_["lag168"])
    mm = LinearRegression().fit(a_.drop(columns="y"), a_["y"])
    ps["ols"].append(pd.Series(mm.predict(b_.drop(columns="y")), index=b_.index))
pv = {k: pd.concat(v) for k, v in ps.items()}
yv = D.loc["2024-04-01":"2024-09-30", "y"]
print(yv.index.equals(pv["ols"].index))
res = pd.DataFrame({k: [mean_squared_error(yv, v) ** 0.5, (abs(yv - v) / yv).mean() * 100] for k, v
in pv.items()}, index=["RMSE", "MAPE"])
print(res.round(2))
```

9b. 更多切分方式

情形	方法	要点
横截面数据，样本独立	`train_test_split(random_state=42)`	随机切
分类且类别不平衡	`stratify=y`	两边类别比例一致
回归，想让 y 分布一致	`stratify=pd.qcut(y, 5, labels=False)`	先分箱再分层
需要 train / valid / test 三份	调用两次 `train_test_split`	0.2 → 再从剩下的里切 0.25，结果是 60/20/20
不想用 sklearn	`sample(frac=0.8)` + `drop(dtr.index)`	效果等同于随机切
时间序列	`shuffle=False` / `iloc` / `loc` 日期 / 布尔条件	先排序，后面的时间做 valid
时间序列，lag 特征跨越边界	在边界处留 gap	防止 valid 的信息通过 lag 漏进 train
同一个 id 有多行（同一客户、同一股票）	`GroupShuffleSplit` / 按 id 抽样	同一个 id 只出现在一边，否则模型会"认出"这个 id

随机切（含分层）

```
dtr, dva = train_test_split(data, test_size=0.2, random_state=42)
dtr, dva = train_test_split(data, test_size=0.2, stratify=data["cls"], random_state=42)
dtr, dva = train_test_split(data, test_size=0.2, stratify=pd.qcut(data["y"], 5, labels=False), rand
om_state=42)
print(dtr["y"].mean(), dva["y"].mean())
```

三份：train / valid / test = 60 / 20 / 20

```
tmp, dte = train_test_split(data, test_size=0.2, random_state=42)
dtr, dva = train_test_split(tmp, test_size=0.25, random_state=42)
print(len(dtr), len(dva), len(dte))
```

用 sample 随机切

```
dtr = data.sample(frac=0.8, random_state=42)
dva = data.drop(dtr.index)
print(len(dtr), len(dva))
```

时间顺序：不打乱、按位置切成三份

```
data = data.sort_values("ts").reset_index(drop=True)
dtr, dva = train_test_split(data, test_size=0.2, shuffle=False)
n1, n2 = int(len(data) * 0.6), int(len(data) * 0.8)
dtr, dva, dte = data.iloc[:n1], data.iloc[n1:n2], data.iloc[n2:]
print(dtr["ts"].max(), dva["ts"].min(), dva["ts"].max(), dte["ts"].min())
```

时间顺序：按分位数或日期切，以及留 gap

```
cut = data["ts"].quantile(0.8)
dtr, dva = data[data["ts"] <= cut], data[data["ts"] > cut]
dtr = data[data["ts"] < "2024-01-01"]
dva = data[(data["ts"] >= "2024-01-01") & (data["ts"] < "2025-01-01")]
d2 = data.set_index("ts")
dtr, dva = d2.loc[:"2023-12-31"], d2.loc["2024-01-01":]
print(len(dtr), len(dva))
gap = 5
dtr, dva = data.iloc[:n1 - gap], data.iloc[n1:]
```

按组切：同一个 id 不同时出现在两边

```
from sklearn.model_selection import GroupShuffleSplit
gs = GroupShuffleSplit(n_splits=1, test_size=0.2, random_state=42)
a, b = next(gs.split(data, groups=data["id"]))
dtr, dva = data.iloc[a], data.iloc[b]
print(len(set(dtr["id"]) & set(dva["id"])))
ids = data["id"].drop_duplicates().sample(frac=0.8, random_state=42)
dtr, dva = data[data["id"].isin(ids)], data[~data["id"].isin(ids)]
print(len(set(dtr["id"]) & set(dva["id"])))
```

两种写法最后都应该打印 0，也就是两边没有共同的 id。

10. 评价指标

回归（顺带算 seasonal naive baseline）

```
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
p = Xva["lag168"]
print(mean_squared_error(yva, p) ** 0.5, mean_absolute_error(yva, p), r2_score(yva, p), (abs(yva - p) / yva).mean() * 100)
```

分类： `accuracy_score`、`roc_auc_score(y, proba[:, 1])`、`f1_score`、`confusion_matrix`、`classification_report(digits=3)`，具体写法见 11.5。

11. 建模

11.1 OLS：在 valid 上评估 → 看系数 → 全量重训 → 预测

```
from sklearn.linear_model import LinearRegression
m = LinearRegression().fit(Xtr, ytr)
pv = m.predict(Xva)
print(mean_squared_error(yva, pv) ** 0.5, r2_score(yva, pv))
coef = pd.Series(m.coef_, index=Xtr.columns)
print(m.intercept_)
print(coef.round(2))
fin = LinearRegression().fit(Xall, yall)
pte = fin.predict(Xte)
```

带 p 值和置信区间的 OLS（需要 statsmodels）

```
import statsmodels.api as sm
o = sm.OLS(ytr, sm.add_constant(Xtr.astype(float))).fit()
print(o.summary())
print(o.params.round(2), o.pvalues.round(3), o.conf_int().round(2), o.rsquared_adj)
```

11.2 Ridge：在 valid 上选 alpha → 全量重训 → 系数换回原始单位

```
from sklearn.linear_model import Ridge, Lasso
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
sc = {}
for a in np.logspace(-2, 4, 25):
    pp = Pipeline([("sc", StandardScaler()), ("m", Ridge(alpha=a))]).fit(Xtr, ytr)
    sc[a] = mean_squared_error(yva, pp.predict(Xva)) ** 0.5
sc = pd.Series(sc)
best = sc.idxmin()
print(best, sc.min())
sc.plot(logx=True, marker="o")
plt.xlabel("alpha")
plt.ylabel("valid RMSE")
plt.show()
fin = Pipeline([("sc", StandardScaler()), ("m", Ridge(alpha=best))]).fit(Xall, yall)
pte = fin.predict(Xte)
s1, rg = fin.named_steps["sc"], fin.named_steps["m"]
coef = pd.DataFrame({"std": rg.coef_, "raw": rg.coef_ / s1.scale_}, index=Xall.columns)
print(rg.intercept_, rg.intercept_ - (rg.coef_ / s1.scale_ * s1.mean_).sum())
print(coef.round(2))
print(coef.reindex(coef["std"].abs().sort_values(ascending=False).index).head(15))
```

- `std` 系数：特征增加 1 个标准差的影响，用来比较连续变量谁更重要。
- `raw` 系数（std ÷ `scale_`）：特征增加 1 个原始单位的影响。dummy 一定要读 raw，意思是"相对于基准类别"。
- 打印的两个截距：第一个是标准化后的截距，等于 y 的均值；第二个是原始单位下的截距。

11.3 Lasso：选 alpha、看哪些系数为 0、看系数路径

```
sc = {}
for a in np.logspace(-1, 3, 25):
    pp = Pipeline([("sc", StandardScaler()), ("m", Lasso(alpha=a, max_iter=10000))]).fit(Xtr, ytr)
    sc[a] = mean_squared_error(yva, pp.predict(Xva)) ** 0.5
sc = pd.Series(sc)
best = sc.idxmin()
print(best, sc.min())
fin = Pipeline([("sc", StandardScaler()), ("m", Lasso(alpha=best, max_iter=10000))]).fit(Xall, yal
l)
pte = fin.predict(Xte)
s1, ls = fin.named_steps["sc"], fin.named_steps["m"]
coef = pd.DataFrame({"std": ls.coef_, "raw": ls.coef_ / s1.scale_}, index=Xall.columns)
print((coef["std"] != 0).sum(), coef.index[coef["std"] == 0].tolist())
```

```
path = {}
for a in np.logspace(0, 4, 20):
    path[a] = Pipeline([("sc", StandardScaler()), ("m", Lasso(alpha=a, max_iter=10000))]).fit(Xtr,
ytr).named_steps["m"].coef_
path = pd.DataFrame(path, index=Xtr.columns).T
path[["lag48", "lag168", "temp_fc", "hol"]].plot(logx=True)
plt.xlabel("alpha")
plt.ylabel("std coef")
plt.show()
```

如果最优 alpha 落在网格边上，就把网格往那一侧扩大再跑一次。

11.4 Random Forest 回归：在 valid 上选参数 → 两种重要性 → 全量重训

```
from sklearn.ensemble import RandomForestRegressor
from sklearn.inspection import permutation_importance
sc = {}
for d in [6, 10, 16, None]:
    for l in [1, 5, 20]:
        rf = RandomForestRegressor(n_estimators=100, max_depth=d, min_samples_leaf=l, max_features=
0.5, n_jobs=-1, random_state=42).fit(Xtr, ytr)
        sc[(d, l)] = mean_squared_error(yva, rf.predict(Xva)) ** 0.5
d, l = min(sc, key=sc.get)
print(sorted(sc.items(), key=lambda x: x[1])[:5], d, l)
rf = RandomForestRegressor(n_estimators=100, max_depth=d, min_samples_leaf=l, max_features=0.5, n_j
obs=-1, random_state=42).fit(Xtr, ytr)
pi = permutation_importance(rf, Xva, yva, n_repeats=3, random_state=42)
imp = pd.DataFrame({"mdi": rf.feature_importances_, "perm": pi.importances_mean}, index=Xva.column
s).sort_values("perm")
imp.tail(15).plot.barh(figsize=(8, 6))
plt.show()
fin = RandomForestRegressor(n_estimators=100, max_depth=d, min_samples_leaf=l, max_features=0.5, n_
jobs=-1, random_state=42).fit(Xall, yall)
pte = fin.predict(Xte)
```

- `pd.Series(sc)` 的 key 是 `(None, 1)` 这种 tuple 时，None 会被转成 NaN，fit 时报 `cannot convert float NaN to integer`。所以用 `min(sc, key=sc.get)` 取最优参数。
- `permutation_importance` 加 `n_jobs=-1` 会撑爆内存，不要加。

11.5 Random Forest 分类：Pipeline + ColumnTransformer

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, roc_auc_score, confusion_matrix, classification_report,
f1_score
from sklearn.inspection import permutation_importance
pre = ColumnTransformer([
    ("n", SimpleImputer(strategy="median"), num),
    ("c", Pipeline([("i", SimpleImputer(strategy="most_frequent")), ("o", OneHotEncoder(handle_unkn
own="ignore"))]), cat),
])
sc = {}
for d in [4, 6, 8, 12, None]:
    for l in [1, 5, 20]:
        p = Pipeline([("p", pre), ("m", RandomForestClassifier(n_estimators=300, max_depth=d, min_s
amples_leaf=l, class_weight="balanced", random_state=42, n_jobs=-1))]).fit(Xtr, ytr)
        sc[(d, l)] = roc_auc_score(yva, p.predict_proba(Xva)[:, 1])
d, l = max(sc, key=sc.get)
print(sorted(sc.items(), key=lambda x: -x[1])[:5])
p = Pipeline([("p", pre), ("m", RandomForestClassifier(n_estimators=300, max_depth=d, min_samples_l
eaf=l, class_weight="balanced", random_state=42, n_jobs=-1))]).fit(Xtr, ytr)
pr = p.predict_proba(Xva)[:, 1]
pd_ = p.predict(Xva)
print(accuracy_score(yva, pd_), roc_auc_score(yva, pr), f1_score(yva, pd_))
print(confusion_matrix(yva, pd_))
print(classification_report(yva, pd_, digits=3))
print(p.score(Xtr, ytr), p.score(Xva, yva))
for t in [0.3, 0.4, 0.5, 0.6]:
    print(t, accuracy_score(yva, (pr >= t).astype(int)))
```

如果题目的评价指标是 accuracy，就把选参数的打分换成 `accuracy_score(yva, p.predict(Xva))`。

特征重要性：permutation（按原始列）+ MDI（按 one-hot 后的列）

```
pi = permutation_importance(p, Xva, yva, n_repeats=5, random_state=42, scoring="roc_auc")
imp = pd.Series(pi.importances_mean, index=Xva.columns).sort_values()
imp.plot.barh()
plt.show()
oh = p.named_steps["p"].named_transformers_["c"].named_steps["o"].get_feature_names(cat).tolist()
mdi = pd.Series(p.named_steps["m"].feature_importances_, index=num + oh).sort_values()
print(mdi.tail(10))
```

全量 train 重训 → 预测 test → 输出 submission

```
fin = Pipeline([("p", pre), ("m", RandomForestClassifier(n_estimators=300, max_depth=d, min_samples
_leaf=l, class_weight="balanced", random_state=42, n_jobs=-1))]).fit(X, y)
sub = pd.DataFrame({"id": test["id"], "out": fin.predict(Xte)})
sub.to_csv("submissions.csv", index=False)
print(sub.shape, sub["out"].value_counts())
```

11.6 Logistic 回归：在 valid 上选 C → 全量重训 → 系数和 odds ratio

```
from sklearn.linear_model import LogisticRegression
pre2 = ColumnTransformer([
    ("n", Pipeline([("i", SimpleImputer(strategy="median")), ("s", StandardScaler())]), num),
    ("c", Pipeline([("i", SimpleImputer(strategy="most_frequent")), ("o", OneHotEncoder(handle_unkn
own="ignore"))]), cat),
])
sc = {}
for C in np.logspace(-3, 2, 11):
    q = Pipeline([("p", pre2), ("m", LogisticRegression(C=C, max_iter=5000))]).fit(Xtr, ytr)
    sc[C] = roc_auc_score(yva, q.predict_proba(Xva)[:, 1])
best = max(sc, key=sc.get)
q = Pipeline([("p", pre2), ("m", LogisticRegression(C=best, max_iter=5000))]).fit(X, y)
oh = q.named_steps["p"].named_transformers_["c"].named_steps["o"].get_feature_names(cat).tolist()
cf = pd.DataFrame({"coef": q.named_steps["m"].coef_[0], "odds": np.exp(q.named_steps["m"].coef_
[0])}, index=num + oh)
print(best, q.named_steps["m"].intercept_, cf.round(3))
```

L1（Lasso 版）logistic：`LogisticRegression(penalty="l1", C=0.05, solver="liblinear")`。`C` 越小，正则化越强。

11.7 GLM（statsmodels）：在 valid 上比 family/link → 全量重训 → summary 和系数

```
import statsmodels.api as sm
from sklearn.metrics import r2_score
F = num
dtr, dva = train_test_split(data, test_size=0.2, random_state=42)
fv = dtr[F].median()
s1 = StandardScaler().fit(dtr[F].fillna(fv))
Xa = sm.add_constant(s1.transform(dtr[F].fillna(fv)))
Xb = sm.add_constant(s1.transform(dva[F].fillna(fv)))
fams = {
    "gauss": sm.families.Gaussian(),
    "logit": sm.families.Binomial(),
    "probit": sm.families.Binomial(sm.families.links.probit()),
    "gamma": sm.families.Gamma(sm.families.links.log()),
}
s = {}
for k, f in fams.items():
    m = sm.GLM(dtr["frac"].values, Xa, family=f).fit()
    s[k] = r2_score(dva["frac"], np.clip(m.predict(Xb), 0, 1))
best = max(s, key=s.get)
print(s, best)
fv = data[F].median()
s2 = StandardScaler().fit(data[F].fillna(fv))
mdl = sm.GLM(data["frac"].values, sm.add_constant(pd.DataFrame(s2.transform(data[F].fillna(fv)), co
lumns=F)), family=fams[best]).fit()
print(mdl.summary())
print(pd.DataFrame({"coef": mdl.params, "p": mdl.pvalues, "exp": np.exp(mdl.params)}).round(3))
pte = np.clip(mdl.predict(sm.add_constant(pd.DataFrame(s2.transform(test[F].fillna(fv)), columns=
F))), 0, 1)
print(pte[:5], mdl.aic, mdl.deviance)
```

- 重训时 scaler 和中位数也要在全量 train 上重新 fit（`s2`），不能沿用 `s1`。
- 系数怎么读：Gaussian 是线性效应；logit 时 `exp(coef)` 是 odds ratio；log link 时 `exp(coef)` 是乘数效应。
- family 怎么选：y 在 (0,1) 用 Binomial；y 恒正且右偏用 Gamma；计数用 Poisson。

二分类 GLM（用 9. 手写处理得到的 a/b）

```
lg = sm.GLM(ytr.values, sm.add_constant(a.astype(float)), family=sm.families.Binomial()).fit()
pb = lg.predict(sm.add_constant(b.astype(float), has_constant="add"))
print(roc_auc_score(yva, pb), accuracy_score(yva, (pb >= 0.5).astype(int)))
```

没有 statsmodels 时用 sklearn 的 GLM

```
from sklearn.linear_model import PoissonRegressor, GammaRegressor, TweedieRegressor
gm = Pipeline([("s", StandardScaler()), ("m", GammaRegressor(alpha=0.0))]).fit(dtr[F].fillna(fv), d
tr["frac"])
print(r2_score(dva["frac"], gm.predict(dva[F].fillna(fv))))
```

12. 残差诊断

按小时、按日类型的 RMSE，残差 vs 预测值，残差 ACF

```
r = yv - pv["ols"]
dt = pd.Series(np.where(D.loc[r.index, "hol"] == 1, "holiday", np.where(r.index.dayofweek >= 5, "we
ekend", "weekday")), index=r.index)
fig, ax = plt.subplots(2, 2, figsize=(14, 8))
(r ** 2).groupby(r.index.hour).mean().pow(0.5).plot.bar(ax=ax[0, 0], title="RMSE by hour")
(r ** 2).groupby(dt).mean().pow(0.5).plot.bar(ax=ax[0, 1], title="RMSE by day type")
ax[1, 0].scatter(pv["ols"], r, s=2, alpha=0.3)
ax[1, 0].axhline(0, color="k")
ax[1, 0].set_title("residual vs predicted")
pd.Series([r.autocorr(k) for k in range(1, 193)], index=range(1, 193)).plot(ax=ax[1, 1], title="res
idual ACF")
plt.tight_layout()
plt.show()
```

最好和最差的一周：预测 vs 实际

```
wk = (r ** 2).groupby(r.index.to_period("W")).mean().pow(0.5)
fig, ax = plt.subplots(2, 1, figsize=(14, 7))
for a_, w in zip(ax, [wk.idxmin(), wk.idxmax()]):
    sl = slice(str(w.start_time.date()), str(w.end_time.date()))
    yv.loc[sl].plot(ax=a_, label="actual")
    pv["ols"].loc[sl].plot(ax=a_, label="forecast")
    a_.set_title(f"{w} RMSE {wk[w]:.0f}")
    a_.legend()
plt.tight_layout()
plt.show()
```

13. VIF（不需要 statsmodels）

```
v = {}
Z = Xtr.astype(float)
for c in ["lag48", "lag168", "temp_fc", "r7"]:
    o_ = Z.drop(columns=c)
    v[c] = 1 / (1 - LinearRegression().fit(o_, Z[c]).score(o_, Z[c]))
print(pd.Series(v).round(1))
```

14. 输出预测文件

```
out = pd.DataFrame({"ts": Xte.index.tz_localize(None).strftime("%Y-%m-%d %H:%M"), "y": np.round(pt
e, 1)})
out.to_csv("forecast.csv", index=False)
print(out.head(), out.shape)
```

15. 看 feature 和 target 的形状，决定要不要变换

思路：先看分布（偏度），再画每个 feature 和 y 的散点图加上分箱均值线，然后用相关系数和单变量 R^2 比较几种变换，最后看多元回归的残差，确认还剩下哪些非线性。下面的例子用一份模拟数据，真实关系是 $\log y = 0.5 \cdot x1 + 0.2 \cdot x2^2 + 0.4 \cdot \log x3 + 0.1 \cdot x4 + 0.5/x5 +$ 噪声。

看到的形状	可能的变换
U 形或倒 U 形	加 `x ** 2`（或者分箱后做 dummy）
一开始陡、后面变平（凹）	`np.log(x)`（x > 0）、`np.log1p(x)`（x ⩾ 0）、`np.sqrt(x)`
越来越陡（凸）	`x ** 2`、`np.exp(x)`，或者对 y 取 log
在 x 很小时剧烈变化	`1 / x`
有台阶 / 阈值	分箱：`pd.cut`、`pd.qcut` 后做 dummy，或者用树模型
y 右偏、和 x 呈指数关系、误差随水平放大（喇叭形）	对 y 取 `np.log(y)` 或 `np.log1p(y)`
斜率随另一个变量的水平而变	交互项 `x1 * x4`，或者按组画图
x 自己右偏得很厉害（skew > 1）	`np.log1p(x)`，能压缩极端值

1. 散点图 + 分箱均值线（上排看 y，下排看 log y）

```
F = ["x1", "x2", "x3", "x4", "x5"]
fig, ax = plt.subplots(2, len(F), figsize=(4 * len(F), 7))
for i, c in enumerate(F):
    ax[0, i].scatter(data[c], data["y"], s=3, alpha=0.3)
    b = pd.qcut(data[c], 20, duplicates="drop")
    m = data.groupby(b)[[c, "y"]].mean()
    ax[0, i].plot(m[c], m["y"], c="r", marker="o")
    ax[0, i].set_title(c)
    ax[1, i].scatter(data[c], np.log(data["y"]), s=3, alpha=0.3)
    m = data.assign(ly=np.log(data["y"])).groupby(b)[[c, "ly"]].mean()
    ax[1, i].plot(m[c], m["ly"], c="r", marker="o")
ax[0, 0].set_ylabel("y")
ax[1, 0].set_ylabel("log y")
plt.tight_layout()
plt.show()
```

红线是按 x 分 20 箱后每箱的平均 y，相当于一条去掉噪声的趋势线。散点很多、看不清的时候，看红线的形状就够了。

2. 分布和偏度

```
fig, ax = plt.subplots(1, len(F) + 1, figsize=(4 * (len(F) + 1), 3))
data["y"].hist(bins=50, ax=ax[0])
ax[0].set_title(f"y skew {data['y'].skew():.2f}")
for i, c in enumerate(F):
    data[c].hist(bins=50, ax=ax[i + 1])
    ax[i + 1].set_title(f"{c} skew {data[c].skew():.2f}")
plt.tight_layout()
plt.show()
print(data[F + ["y"]].skew().round(2))
```

|skew| > 1 算是明显偏斜。

3. 用相关系数比较几种变换

```
ly = np.log(data["y"])
res = {}
for c in F:
    x = data[c]
    res[c] = {
        "x": np.corrcoef(x, ly)[0, 1],
        "x2": np.corrcoef(x ** 2, ly)[0, 1],
        "log": np.corrcoef(np.log(x), ly)[0, 1] if (x > 0).all() else np.nan,
        "sqrt": np.corrcoef(np.sqrt(x), ly)[0, 1] if (x >= 0).all() else np.nan,
        "inv": np.corrcoef(1 / x, ly)[0, 1] if (x != 0).all() else np.nan,
        "exp": np.corrcoef(np.exp(x), ly)[0, 1],
        "spear": x.corr(ly, method="spearman"),
    }
res = pd.DataFrame(res).T.round(3)
print(res)
print(res.drop(columns="spear").abs().idxmax(axis=1))
```

模拟数据的结果：

	x	x^2	log	sqrt	1/x	spearman	结论
x1	0.531	0.014	—	—	0.011	0.526	线性
x2	−0.000	**0.550**	—	—	−0.029	0.002	U 形：线性相关和 Spearman 都约等于 0，平方后相关变得很高
x3	0.314	0.164	**0.414**	0.385	−0.330	0.401	取 log
x4	0.085	0.080	0.078	0.086	−0.000	0.074	都差不多，保持原样
x5	−0.263	−0.211	−0.335	−0.299	**0.373**	−0.243	取 1/x

- Pearson 只看线性关系，Spearman 看单调关系。Spearman 明显比 Pearson 高，说明关系是单调但非线性的，可以考虑 log、sqrt 这类变换。
- 两者都约等于 0、但散点图有明显形状（比如 x2），说明是非单调关系，要考虑平方或分箱。
- 几种变换差不多时（比如 x4），保留原样，不要为了一点点提升增加复杂度。

4. 单变量 R^2：线性 vs 二次 vs log

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score
r2 = {}
for c in F:
    x = data[[c]]
    r2[(c, "linear")] = LinearRegression().fit(x, ly).score(x, ly)
    xq = pd.concat([data[c], data[c] ** 2], axis=1)
    r2[(c, "quad")] = LinearRegression().fit(xq, ly).score(xq, ly)
    if (data[c] > 0).all():
        xl = np.log(data[[c]])
        r2[(c, "log")] = LinearRegression().fit(xl, ly).score(xl, ly)
print(pd.Series(r2).unstack().round(3))
```

quad 同时包含 x 和 x^2，所以它的 R^2 一定 ≥ linear。只有提升明显时（比如 x2：0.000 → 0.302）才说明真有曲率。

5. 多元回归后，残差 vs 每个 feature

```
m = LinearRegression().fit(data[F], ly)
e = ly - m.predict(data[F])
fig, ax = plt.subplots(1, len(F), figsize=(4 * len(F), 3))
for i, c in enumerate(F):
    ax[i].scatter(data[c], e, s=3, alpha=0.3)
    b = pd.qcut(data[c], 20, duplicates="drop")
    mm = pd.DataFrame({"x": data[c], "e": e}).groupby(b).mean()
    ax[i].plot(mm["x"], mm["e"], c="r")
    ax[i].axhline(0, c="k")
    ax[i].set_title(c)
plt.tight_layout()
plt.show()
```

红线贴着 0 而且是平的，说明这个 feature 的线性项已经够了；红线弯曲，说明还缺一个非线性项。这一步控制了其他 feature 的影响，比单变量散点图更可靠。

6. 做变换，并和不变换比较

```
X = data[F].copy()
X["x2sq"] = X["x2"] ** 2
X["lx3"] = np.log(X["x3"])
X["ix5"] = 1 / X["x5"]
X = X.drop(columns=["x3", "x5"])
y = np.log(data["y"])
a, b = data.index[:1600], data.index[1600:]
m0 = LinearRegression().fit(data.loc[a, F], data.loc[a, "y"])
m1 = LinearRegression().fit(X.loc[a], y.loc[a])
p1 = np.exp(m1.predict(X.loc[b]))
print(r2_score(data.loc[b, "y"], m0.predict(data.loc[b, F])), r2_score(data.loc[b, "y"], p1))
```

- 模拟数据上，原始尺度的 valid R^2 从 0.30 提升到 0.91。
- 对 y 取 log 训练之后，预测要用 `np.exp` 变回原始尺度，再和原始的 y 比较打分；`log1p` 对应的是 `expm1`。
- log、平方、1/x 都是逐行变换，不涉及统计量，在切分之前做也不会泄漏。用到均值或分位数的变换（标准化、分箱边界），必须只在 train 上 fit。
- 树模型对单调变换（log、sqrt）不敏感，不需要做；线性模型和 GLM 才需要。

7. 检查交互：按第三个变量分组画图

```
print(np.corrcoef(data["x1"] * data["x4"], ly)[0, 1])
data["b4"] = pd.qcut(data["x4"], 3, labels=["lo", "mid", "hi"])
for k, gg in data.groupby("b4"):
    plt.scatter(gg["x1"], np.log(gg["y"]), s=3, alpha=0.4, label=k)
plt.legend()
plt.show()
```

三组的斜率明显不同时，就加交互项 `x1 * x4`。

16. 5-fold cross validation

5-fold CV 的过程：

1. 把训练数据分成 5 份。
2. 每一轮拿其中 1 份做验证，其余 4 份训练，一共 5 轮。
3. 得到 5 个分数，报告均值和标准差。
4. 用 CV 均值最好的参数，在全部训练数据上重训一次。
5. 最后去预测 test。

预处理（填缺失、标准化、编码）要放进 Pipeline，这样每一折都只在那一折的训练部分 fit，验证部分没有泄漏。

切法	写法	适用
打乱后切 5 份	`KFold(5, shuffle=True, random_state=42)`	横截面数据，行与行独立
不打乱，按顺序切成 5 段	`KFold(5, shuffle=False)`	数据本身没有排序时可以用。**注意：** 它会拿后面的数据训练、去预测前面那段，所以对时间序列仍然有 look-ahead
分层	`StratifiedKFold(5, shuffle=True, random_state=42)`	分类问题，保证每折的类别比例一致
时间序列	`TimeSeriesSplit(5)`	只用过去训练、预测未来；训练集是扩展窗口
按组	`GroupKFold(5)` + `groups=`	同一个 id 不会同时出现在训练和验证里

1. 手写 5-fold 循环（shuffle=True）

```
from sklearn.model_selection import KFold, StratifiedKFold, TimeSeriesSplit, GroupKFold, cross_val_
score, cross_validate
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_squared_error
Xc = X.reset_index(drop=True)
yc = y.reset_index(drop=True)
kf = KFold(n_splits=5, shuffle=True, random_state=42)
sc = []
for i, (a, b) in enumerate(kf.split(Xc)):
    m = Pipeline([("s", StandardScaler()), ("m", Ridge(alpha=1.0))]).fit(Xc.iloc[a], yc.iloc[a])
    sc.append(mean_squared_error(yc.iloc[b], m.predict(Xc.iloc[b])) ** 0.5)
    print(i, len(a), len(b), round(sc[-1], 4))
print(np.mean(sc), np.std(sc))
```

`kf.split` 每次返回两组**行号**：`a` 是训练行，`b` 是验证行，所以要用 `iloc` 取数据。

2. shuffle=False：每折是连续的一段

```
kf = KFold(n_splits=5, shuffle=False)
for i, (a, b) in enumerate(kf.split(Xc)):
    print(i, a.min(), a.max(), b.min(), b.max())
```

输出依次是验证段 0–399、400–799……第 1 折的训练集用的是 400–1999，也就是用"未来"预测"过去"。所以时间序列要用 TimeSeriesSplit。

3. 一行写法：cross_val_score / cross_validate

```
kf = KFold(n_splits=5, shuffle=True, random_state=42)
p = Pipeline([("s", StandardScaler()), ("m", Ridge(alpha=1.0))])
s = cross_val_score(p, Xc, yc, cv=kf, scoring="neg_root_mean_squared_error")
print(-s, -s.mean(), s.std())
s = cross_validate(p, Xc, yc, cv=kf, scoring=["r2", "neg_mean_absolute_error"], return_train_score=
True)
print(pd.DataFrame(s).round(3))
```

- sklearn 规定分数越大越好，所以误差类指标都带 `neg_` 前缀，取负号才是 RMSE。
- `cv=5` 直接写数字时，默认**不打乱**：回归用 KFold，分类用 StratifiedKFold。想打乱必须自己传 `KFold(..., shuffle=True)`。
- `return_train_score=True` 会同时给出训练分数。训练分数远好于验证分数，说明过拟合。

4. 用 CV 选参数，再在全部数据上重训

```
res = {}
for al in np.logspace(-3, 3, 13):
    p = Pipeline([("s", StandardScaler()), ("m", Ridge(alpha=al))])
    s = -cross_val_score(p, Xc, yc, cv=KFold(5, shuffle=True, random_state=42), scoring="neg_root_mean_squared_error")
    res[al] = [s.mean(), s.std()]
res = pd.DataFrame(res, index=["mean", "std"]).T
best = res["mean"].idxmin()
print(res.round(4), best)
fin = Pipeline([("s", StandardScaler()), ("m", Ridge(alpha=best))]).fit(Xc, yc)
```

所有参数都必须用同一个 `random_state` 的 KFold，保证大家在同样的 5 折上比较。几组参数的均值差距小于 std 时，它们其实分不出好坏，选更简单（正则化更强）的那组。

5. 分类：StratifiedKFold

```
yb = (yc > yc.median()).astype(int)
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
for a, b in skf.split(Xc, yb):
    m = Pipeline([("s", StandardScaler()), ("m", LogisticRegression(max_iter=1000))]).fit(Xc.iloc[a], yb.iloc[a])
    print(round(yb.iloc[b].mean(), 3), round(roc_auc_score(yb.iloc[b], m.predict_proba(Xc.iloc[b])[:, 1]), 4))
print(cross_val_score(Pipeline([("s", StandardScaler()), ("m", LogisticRegression(max_iter=1000))]), Xc, yb, cv=skf, scoring="roc_auc").mean())
```

`split(X, y)` 一定要传 y，它要根据 y 来分层。

6. 时间序列：TimeSeriesSplit（不能 shuffle）

```
tss = TimeSeriesSplit(n_splits=5)
for i, (a, b) in enumerate(tss.split(Xc)):
    m = Pipeline([("s", StandardScaler()), ("m", Ridge(alpha=1.0))]).fit(Xc.iloc[a], yc.iloc[a])
    print(i, a.min(), a.max(), b.min(), b.max(), round(mean_squared_error(yc.iloc[b], m.predict(Xc.iloc[b])) ** 0.5, 4))
tss = TimeSeriesSplit(n_splits=5, max_train_size=500)
print(-cross_val_score(Pipeline([("s", StandardScaler()), ("m", Ridge())]), Xc, yc, cv=tss, scoring="neg_root_mean_squared_error"))
```

- 数据必须先按时间排好序。第 i 折用 0 到某个时点的数据训练，预测紧接着的下一段。
- `max_train_size` 把训练集限制成滚动窗口，不设就是扩展窗口。
- 新版 sklearn 有 `gap=` 参数，可以在训练和验证之间留空；0.23 没有这个参数。

7. 按组：GroupKFold

```
gk = GroupKFold(n_splits=5)
print(-cross_val_score(Pipeline([("s", StandardScaler()), ("m", Ridge())]), Xc, yc, cv=gk, groups=data["id"].values, scoring="neg_root_mean_squared_error"))
```

`scoring=` 常用写法	含义
`"neg_root_mean_squared_error"` / `"neg_mean_squared_error"` / `"neg_mean_absolute_error"`	回归误差，取负号后才是正数
`"r2"`	R^2
`"accuracy"` / `"roc_auc"` / `"f1"` / `"neg_log_loss"`	分类指标

17. 各模型的输出、适用场景和结果解读

17.1 模型对照

模型	适用	fit 后能拿到的结果	怎么解读
OLS `LinearRegression`	连续 y，关系大致线性，n 远大于 p；需要可解释的系数	`coef_`、`intercept_`、`predict`、`score`（R^2）	系数：x 增加 1 个单位，y 平均变化 coef（其他变量不变）。dummy 系数表示相对基准类别的差值
OLS `sm.OLS`	同上，还需要显著性	`params`、`bse`、`tvalues`、`pvalues`、`conf_int()`、`rsquared`、`rsquared_adj`、`aic`、`bic`、`summary()`	系数同上；$p < 0.05$ 说明显著不等于 0；置信区间不包含 0 也表示显著
Ridge	特征多、共线性强，需要稳定的系数	同 OLS（没有 p 值）；要先标准化	系数被往 0 压缩。标准化系数可以比较重要性；除以 `scale_` 换回原始单位。alpha 越大压缩越强
Lasso	特征多，想自动挑出重要的特征	同 Ridge；有一部分 `coef_` 正好等于 0	系数为 0 表示被剔除。共线性很强的一组特征里，Lasso 往往只留一个，所以"被剔除"不等于"没有用"
Logistic `LogisticRegression` / GLM Binomial	二分类，或 0–1 之间的比例；要概率和可解释性	`predict_proba`、`predict`、`coef_`、`intercept_`（statsmodels 还有 p 值）	系数单位是 log-odds；`exp(coef)` 是 odds ratio，比如 1.5 表示 odds 增加 50%。`C` 越小正则化越强
GLM Gamma（log link）	$y > 0$、右偏，方差随均值变大（金额、时长）	`params` / `coef_`	`exp(coef)` 是乘数效应：x 增加 1，y 变成原来的 exp(coef) 倍，也就是变化 $(\exp - 1)\times 100\%$
GLM Poisson（log link）	计数 y（次数、件数）	同上	同上，是对发生率的乘数效应
RandomForest 回归 / 分类	非线性、有交互、类别和数值混在一起，不想做变换；对单调变换和离群值不敏感	`predict`、`predict_proba`、`feature_importances_`（MDI）、`estimators_`；permutation importance	**没有系数**，只看重要性（哪个 feature 影响大，但不知道方向）。看方向要画 partial dependence。**不能外推**：预测值不会超出训练集里 y 的范围，有趋势的时间序列要注意

17.2 回归指标

指标	公式 / 含义	好坏	注意
RMSE	$\sqrt{\text{mean}((y-p)^2)}$，单位和 y 一样，可以读作"典型误差有多大"	越小越好	对大误差敏感
MAE	$\text{mean}(\lvert y-p\rvert)$，平均绝对误差	越小越好	对离群值更稳健；RMSE 明显大于 MAE，说明有少数大误差
MAPE	$\text{mean}(\lvert y-p\rvert / y) \times 100\%$，平均百分比误差	越小越好	无单位，方便跨数据集比较；y 接近 0 时会爆掉
R^2	$1 - \text{SSR}/\text{SST}$：模型解释了 y 的波动的百分之多少	1 最好；0 等于一直猜均值；可以为负	时间序列里 R^2 很容易很高，要再和 naive baseline 比较
Adj R^2	对特征个数做了惩罚的 R^2	越大越好	用来比较特征个数不同的模型
AIC / BIC	$-2 \cdot \log L$ + 惩罚项（BIC 的惩罚更重）	越小越好	只能比较同一份 y、同一个 family 的模型
Deviance（GLM）	模型和完美拟合之间的差距，相当于 GLM 版的残差平方和	越小越好	不同 family 之间不能比

指标	公式 / 含义	好坏	注意
相对 baseline 的提升	1 − RMSE_model / RMSE_naive	越大越好	时间序列一定要报这个，naive 通常是 lag24 或 lag168

17.3 分类指标

指标	含义	什么时候看
Confusion matrix	`[[TN, FP], [FN, TP]]`，行是真实类别，列是预测类别	先看这个，知道错在哪一类
Accuracy	(TP + TN) / 全部	类别平衡时可以看。不平衡时会骗人：90% 是 0 的话，全猜 0 也有 0.9
Precision	TP / (TP + FP)：预测为 1 的样本里，有多少真的是 1	误报代价大的时候（比如发出警报要花钱）
Recall	TP / (TP + FN)：真实是 1 的样本里，被抓到了多少	漏报代价大的时候（比如流失、欺诈）
F1	Precision 和 Recall 的调和平均	想同时兼顾两者，类别不平衡时
ROC-AUC	随机抽一个正例和一个负例，模型给正例打分更高的概率	不依赖阈值，看整体排序能力。0.5 等于随机，0.7–0.8 是一般业务水平，>0.9 时要先检查有没有泄漏
PR-AUC `average_precision_score`	Precision-Recall 曲线下的面积	正例非常少时比 ROC-AUC 更诚实。它的基准线是正例的占比，不是 0.5
Log loss	−mean(y · log p + (1−y) · log(1−p))	越小越好；衡量的是概率准不准，而不只是排序
`classification_report`	每个类别的 precision、recall、f1、support（样本数），以及 macro / weighted 平均	一次看全

17.4 statsmodels summary 怎么读

字段	含义
coef	系数估计值
std err	系数的标准误，表示估计有多不确定
t（OLS）/ z（GLM）	coef / std err；绝对值大于 2 大致就是显著
P>|t|	原假设"系数 = 0"成立时，看到这么大的 t 的概率。小于 0.05 叫显著
[0.025, 0.975]	95% 置信区间；不包含 0 就是显著
R-squared / Adj. R-squared	拟合优度，见 17.2
F-statistic / Prob(F)	检验所有系数是否都等于 0；Prob 很小说明模型整体有用
Log-Likelihood / AIC / BIC	用来比较模型，AIC、BIC 越小越好
Durbin-Watson	残差的一阶自相关。约等于 2 表示没有自相关；明显小于 2 是正自相关，时间序列很常见，这时标准误会被低估
Omnibus / Jarque-Bera (JB)	检验残差是否服从正态；p 很小说明不是正态。样本大时几乎总是拒绝，影响不大
Skew / Kurtosis	残差的偏度和峰度（正态分布分别是 0 和 3）
Cond. No.	条件数；很大（> 1000，或者 summary 给出警告）说明有共线性或没有标准化
Deviance / Pearson chi2（GLM）	拟合差距；Pearson chi2 / 自由度远大于 1，说明存在过度离散

17.5 系数和重要性怎么讲

结果	讲法
原始单位系数	"其他变量不变，temperature 高 1 °C，load 平均低 82 MW"
标准化系数	"x 增加 1 个标准差，y 变化多少"；用来比较不同连续变量谁的影响大
dummy 系数	"周六比基准类别（周一）低 X"，一定要说清楚基准类别是什么
交互项	要把相关的几项加起来讲，比如"周末 10 点 = 小时项 + 星期项 + 交互项"
log(y) 模型的系数	x 增加 1，y 大约变化 coef × 100%（coef 较小时）；准确值是 (e^coef − 1) × 100%
log(x) 的系数	x 增加 1%，y 变化 coef / 100 个单位；如果 y 也取了 log，coef 就是弹性
odds ratio	"x 增加 1 个标准差，流失的 odds 是原来的 1.5 倍"
MDI vs permutation	permutation 是在 valid 上把这一列打乱后掉了多少分，代表泛化重要性，汇报时以它为准。MDI 偏向取值多的连续变量
共线性	单个系数不稳定（VIF > 10），按组解释；对预测本身影响不大
CV 均值 ± std	均值是预期表现，std 是稳定性。两个模型的差距小于 std 时，不能说哪个更好
train vs valid 分数	train 远好于 valid 是过拟合；两者都差是欠拟合；valid 明显好于 train，要检查有没有泄漏

18. 常见 method 一览（每个位置放什么变量）

占位符说明

占位符	是什么	例子
`df`	DataFrame	`data`、`Xtr`
`s`	Series（一列）	`df["y"]`、`ytr`
`idx`	Index / DatetimeIndex（行标签）	`df.index`
`"c"`	一个列名（字符串）	`"y"`
`["c1","c2"]`	列名的 list	`["sec", "reg"]`
`cond`	布尔 Series（True/False，和 df 一样长）	`df["y"] > 0`
`f`	函数，或函数名字符串	`"mean"`、`np.log`、`lambda x: x.max()`
`"freq"`	频率字符串	`"H"` 小时、`"D"` 天、`"W"` 周、`"M"` 月
`n` `k`	整数	`168`、`5`
`q`	0–1 之间的小数	`0.2`、`0.99`
`a`	numpy 数组	`m.predict(X)`
`X` / `y`	特征表 / 目标列（DataFrame 或数组 / Series 或数组）	`Xtr`、`ytr`
`m`	sklearn 或 statsmodels 的模型对象	`Ridge(alpha=1)`
`{k: v}`	字典	`{"yes": 1}`

18.1 读取、查看

写法	各位置	返回
`pd.read_csv("path", parse_dates=["c"], index_col="c")`	文件路径；要解析成时间的列名 list；作为 index 的列名	df
`df.head(n)` / `df.tail(n)` / `df.sample(n, random_state=k)`	行数	df

写法	各位置	返回
`df.shape` / `df.columns` / `df.index` / `df.dtypes`	属性，不加括号	tuple / Index / Series
`df.info()` / `df.describe(include="all")`	`"all"` 表示连非数值列也一起统计	打印 / df
`df.isna().sum()` / `df.isna().mean()`	—	s：每列缺失个数 / 缺失比例
`df.duplicated(subset=["c"]).sum()`	用哪些列判断重复	整数
`s.value_counts(dropna=False, normalize=True)`	是否把 NaN 算进去；是否返回比例	s
`s.unique()` / `s.nunique()`	—	数组 / 整数
`s.quantile(q)` / `s.quantile([q1, q2])`	一个或多个分位点	数 / s
`df.corr(method="pearson")` / `s1.corr(s2)`	`"pearson"` 或 `"spearman"`；另一个 Series	df / 数
`s.autocorr(n)`	lag 阶数	数
`df.select_dtypes("number")` / `("object")`	dtype 名	df

18.2 选取、修改

写法	各位置	返回
`df["c"]` / `df[["c1","c2"]]`	列名 / 列名 list	s / df
`df[cond]`	布尔 Series	df（筛行）
`df.loc[行, 列]`	行：标签、标签切片 `"2024-01":"2024-03"` 或 cond；列：列名或 list	按**标签**取，切片两端都包含
`df.iloc[行号, 列号]`	整数或整数切片 `:n`	按**位置**取，右端不包含
`df.loc[cond, "c"] = v`	条件；要改的列；新值	原地修改
`df.drop(columns=["c"])` / `df.drop(index=idx)`	要删的列 / 要删的行标签	df
`df.rename(columns={"旧": "新"})` / `s.rename("新")`	字典 / 新名字	df / s
`df.assign(new=s)`	新列名 = 值	df
`df.set_index("c")` / `df.reset_index(drop=True)`	列名 / 是否丢掉旧 index	df
`df.sort_values(["c1","c2"], ascending=[True, False])`	排序列；每列升序还是降序	df
`df.astype({"c": int})` / `s.astype(float)`	字典或类型	df / s
`df.copy()`	—	独立副本

18.3 清洗

写法	各位置	返回
`df.drop_duplicates(subset=["c"], keep="first")`	判断用的列；保留第一个或最后一个	df
`df.dropna(subset=["c"], how="any")`	只看这些列；`"any"` 有一个缺失就删，`"all"` 全缺才删	df

写法	各位置	返回
`df.fillna(v)` / `fillna({"c": v})` / `fillna(df2)`	一个值 / 每列一个值 / 按位置对齐的另一张表	df
`df.ffill(limit=n)` / `df.bfill()`	最多连续填几个	df
`s.interpolate()`	—	s（注意会用到后面的值，有 look-ahead）
`s.clip(lo, hi)` / `df.clip(s_lo, s_hi, axis=1)`	下界、上界；按列给界时传 Series	s / df
`s.between(lo, hi)`	下界、上界（两端都包含）	布尔 s
`s.isin([v1, v2])`	值的 list	布尔 s
`s.map({k: v})` / `s.map(f)`	字典（查不到变 NaN）/ 函数	s
`s.replace({旧: 新})` / `s.replace("re", "", regex=True)`	字典（查不到保留原值）/ 正则	s
`pd.to_numeric(s, errors="coerce")`	要转换的列；转不了就变 NaN	s
`pd.to_datetime(s, format="%Y-%m-%d", errors="coerce")`	列；格式模板	s（datetime）
`s.str.strip()` `.str.lower()` `.str.replace("a", "b", regex=False)`	旧串、新串	s
`s.str.contains("pat", na=False)` / `s.str.match(r"^\d+$", na=False)`	子串或正则；NaN 当作 False	布尔 s
`s.where(cond, other)` / `s.mask(cond, other)`	条件；不满足（where）/ 满足（mask）时换成 other	s
`np.where(cond, a, b)`	条件；真时取的值；假时取的值	数组

18.4 时间

写法	各位置	返回
`s.dt.tz_localize("Europe/Berlin", ambiguous="infer")`	时区名；秋季重复时刻怎么处理	s
`s.dt.tz_convert("UTC")` / `idx.tz_convert(...)`	目标时区	s / idx
`idx.tz_localize(None)`	去掉时区标签	idx
`df.asfreq("freq")`	频率（index 必须是时间）	df，缺的时刻补成 NaN 行
`df.resample("freq").mean()` / `.agg({"c": f})`	频率；聚合函数	df
`s.rolling(n).mean()` / `.std()`	窗口长度	s
`s.shift(n)` / `s.diff(n)` / `s.pct_change(n)`	往下挪 n 行；n 为负表示往上（有 look-ahead）	s
`pd.date_range("start", "end", freq="freq")`	起点、终点、频率	idx
`pd.period_range("2024-04", "2024-09", freq="M")`	起止、频率	一组 Period
`idx.hour` `.dayofweek` `.month` `.year` `.date`	属性，不加括号	数组
`s.dt.hour` / `s.dt.days` / `s.dt.strftime("%Y-%m-%d %H:%M")`	是列（不是 index）时要加 `.dt`	s
`idx.to_period("W")`	频率	每个时刻所在的周或月
`pd.Timestamp("2024-01-01")` / `pd.Timedelta(days=n)`	时间点 / 时间长度	标量

18.5 分组、拼接、重塑

写法	各位置	返回
`df.groupby(["c1","c2"])["c"].f()`	分组键（列名、list，或和 df 一样长的数组）；要算的列；聚合函数	s（MultiIndex）
`df.groupby("c").agg(new=("c2", f))`	新列名 = (原列, 函数)	df
`df.groupby("c")["c2"].transform(f)`	函数	s，**和 df 一样长**
`df.pivot_table(index="c1", columns="c2", values="c3", aggfunc=f)`	行、列、值、聚合函数	df
`pd.crosstab(s1, s2, normalize="index")`	行变量、列变量	df
`s.unstack()` / `df.stack()`	默认最内层	df / s
`df.melt(id_vars=["c"], var_name="v", value_name="val")`	保持不动的列；新的两列名	df（长表）
`pd.concat([df1, df2], axis=1)`	要拼的 list；0 上下堆叠，1 左右拼接（按 index 对齐）	df
`df1.join(df2, how="outer")`	另一张表；按 **index** 合并	df
`df1.merge(df2, on=["c"], how="left")`	另一张表；按**列**合并的键	df
`s.reindex(idx, fill_value=v)` / `df.reindex(columns=["c"], fill_value=0)`	新的行标签 / 列名	按新标签对齐
`pd.get_dummies(s, prefix="p", drop_first=True)` / `pd.get_dummies(df, columns=["c"], dtype=int)`	要编码的列；列名前缀；是否丢掉基准类别	df
`pd.cut(s, bins=[...])` / `pd.qcut(s, n, labels=False)`	给定边界 / 等分成 n 箱	s（类别）
`df.mul(s, axis=0)`	一列；`axis=0` 表示按行对齐	df

18.6 numpy

写法	各位置	返回
`np.log(a)` / `np.log1p(a)` / `np.exp(a)` / `np.expm1(a)` / `np.sqrt(a)`	数组或 Series	同形状
`np.clip(a, lo, hi)`	数组、下界、上界	数组
`np.corrcoef(a1, a2)[0, 1]`	两个数组	2×2 矩阵，取 [0, 1]
`np.logspace(lo, hi, n)`	10^lo 到 10^hi，取 n 个点	数组
`np.linspace(lo, hi, n)` / `np.arange(lo, hi, step)`	n 个等间距点 / 按步长（不含 hi）	数组
`np.mean(a, axis=0)` / `np.std(a)` / `np.percentile(a, [1, 99])`	0 按列算，1 按行算	数 / 数组
`np.r_[a1, a2]`	首尾拼接	数组
`a[:, 1]` / `a[行, 列]`	二维切片	数组
`np.random.RandomState(k).normal(mu, sd, n)`	种子；均值、标准差、个数	数组

18.7 matplotlib / pandas 画图

写法	各位置	说明
`fig, ax = plt.subplots(r, c, figsize=(w, h), sharey=True)`	行数、列数、整张图的大小（英寸）	ax 是数组：`ax[i, j]`；只有一行时是 `ax[j]`

写法	各位置	说明
`s.plot(ax=ax[0], label="x", title="t", alpha=q)` / `df.plot()`	画在哪个子图；图例名；标题；透明度	df 每列一条线
`s.plot.bar()` / `.barh()` / `.hist(bins=n)` / `df.boxplot(column="c", by=s)`	—	条形 / 横条 / 直方 / 箱线
`plt.scatter(x, y, s=n, alpha=q, label="g")`	x 数组、y 数组、点的大小、透明度、图例名	散点
`plt.plot(x, y, c="r", ls="--", marker="o")`	x、y、颜色、线型、点形	折线
`plt.imshow(df, aspect="auto", cmap="viridis")` + `plt.colorbar()`	二维表、长宽比、色系	热力图
`plt.axhline(v)` / `plt.axvline(v, ls="--")`	位置	参考线
`plt.xticks(位置list, 标签list)` / `plt.xlabel("x")` / `plt.title("t")` / `plt.legend()`	—	坐标和标注
`ax.set_title("t")` / `ax.set_xlabel("x")`	子图用 `set_` 开头的方法	—
`plt.tight_layout()` / `plt.show()`	—	自动排版 / 显示

18.8 sklearn：切分和预处理

写法	各位置	返回
`train_test_split(X, y, test_size=q, stratify=y, shuffle=True, random_state=k)`	要切的一个或多个表；验证集比例；按什么分层；是否打乱；种子	按输入顺序返回：`Xtr, Xva, ytr, yva`
`KFold(n_splits=k, shuffle=True, random_state=k)`	折数、是否打乱、种子	切分器；`.split(X)` 每次给出 (训练行号, 验证行号)
`StratifiedKFold(k, shuffle=True)` / `TimeSeriesSplit(k)` / `GroupKFold(k)`	折数	`.split(X, y)` / `.split(X)` / `.split(X, y, groups=s)`
`cross_val_score(m, X, y, cv=切分器, scoring="r2")`	模型、特征、目标、切分器、指标名	每折分数的数组
`StandardScaler().fit(Xtr)` → `.transform(X)`	fit 只用训练集；transform 用于任何数据	数组（列名丢失）
`SimpleImputer(strategy="median")`	`"mean"`、`"median"`、`"most_frequent"`	转换器
`OneHotEncoder(handle_unknown="ignore")`	遇到新类别时编码成全 0	转换器
`ColumnTransformer([("名", 转换器, 列名list), ...])`	每项：名字、对哪些列、用什么处理	转换器
`Pipeline([("名", 步骤), ..., ("m", 模型)])`	按顺序列出各步骤，最后一步是模型	模型；取某一步用 `p.named_steps["名"]` 或 `p["名"]`
`p.fit(X, y, m__sample_weight=w)`	给名叫 `m` 的步骤传参数：步骤名 + 两个下划线 + 参数名	—

18.9 sklearn：模型和指标

写法	各位置	拿结果
`LinearRegression().fit(X, y)`	特征、目标	`.coef_` `.intercept_` `.predict(X)` `.score(X, y)`

写法	各位置	拿结果
`Ridge(alpha=v)` / `Lasso(alpha=v, max_iter=n)`	正则化强度	同上
`LogisticRegression(C=v, penalty="l2", max_iter=n)`	C 越小正则化越强	`.predict_proba(X)[:, 1]` `.predict(X)` `.coef_[0]`
`TweedieRegressor(power=0, link="identity", alpha=0)` / `GammaRegressor(alpha=0)` / `PoissonRegressor(alpha=0)`	`alpha=0` 表示不加惩罚	`.coef_` `.predict`
`RandomForestRegressor(n_estimators=n, max_depth=k, min_samples_leaf=k, max_features=q, n_jobs=-1, random_state=k)`	树的数量；最大深度（None 表示不限）；叶子最少样本数；每次分裂考虑多少比例的特征	`.predict` `.feature_importances_`
`RandomForestClassifier(..., class_weight="balanced")`	同上，再给少数类加权	`.predict_proba(X)[:, 1]`
`permutation_importance(m, Xva, yva, n_repeats=k, scoring="r2")`	已经训练好的模型、验证集、重复次数	`.importances_mean`
`mean_squared_error(y, p) ** 0.5` / `mean_absolute_error(y, p)` / `r2_score(y, p)`	**真实值在前，预测值在后**	数
`accuracy_score(y, p)` / `f1_score(y, p)` / `confusion_matrix(y, p)` / `classification_report(y, p)`	真实、预测类别	数 / 矩阵 / 文本
`roc_auc_score(y, prob)`	真实、**概率**	数

18.10 statsmodels

写法	各位置	拿结果
`sm.add_constant(X, has_constant="add")`	特征；强制加截距列	df 或数组，多一列 `const`
`sm.OLS(y, X).fit()`	**y 在前，X 在后**（和 sklearn 相反）	`r.params` `r.pvalues` `r.bse` `r.conf_int()` `r.rsquared` `r.summary()`
`sm.GLM(y, X, family=sm.families.Binomial()).fit()`	目标、特征、分布 family	同上，另有 `r.aic` `r.deviance`
`sm.families.Binomial(sm.families.links.probit())`	在 family 里换 link；旧版 link 名是小写	—
`r.predict(X_new)`	新数据，也要先 `add_constant`	数组